

# Optimizing Farkle Strategy through Reinforcement Learning and Markov Decision Processes

Nathan Cohen & Robert Kaplan  
*Department of Aerospace Engineering*  
*University of Colorado at Boulder*  
Boulder, CO, United States of America

**Abstract**—This paper examines the utilization of Reinforcement Learning (RL) in the stochastic, multi-dice game Farkle. We find the best policy for the game by modeling its complicated decision space as a Markov Decision Process (MDP) and using Tabular Q-Learning. This policy changes based on the immediate competitive context to find the best balance between risk and reward. To lessen the curse of dimensionality, we present a state-space discretization method that combines score differentials, significantly cutting down on the amount of computing power needed. Our findings indicate that the trained RL agent substantially surpasses conventional heuristic baselines by effectively adjusting its risk tolerance as the game nears terminal states.

**Index Terms**—Reinforcement Learning, Q-Learning, Markov Decision Process, Stochastic Games, Farkle.

## I. INTRODUCTION

There are many types of games out there which can be categorized in many different ways such as dice, card, board, video, etc. Each style of game, and each game within each category, has some form of choice based operations that allow for the continuation of the game. Thus, each game can be analyzed and a predictive algorithm can be made in order to choose the best next choice in the game. The dice-based game of chance known as Farkle is no different. However, since it is a dice-based game, it has inherit randomness built into the play of the game which limit the player's ability to make certain predictions unlike in games like Chess or Go. This results in a state space where the risk-reward scenarios are constantly fluctuating. This is further compounded when the game is expanded to include more than just one player as this adds incentive to not just play safely and with minimal risk. All of this culminates in a policy that ultimately changes not only based on the current state of the player for the round, but also must account for how the player compares to all others at the moment and in future rounds.

While human players may have an advantage against a purely random player, there is still a high level of variance in their choices which can ultimately be flawed. Reinforcement Learning can be used to systematically derive policies for a virtual player that maximize the probability of winning rather than purely maximizing their expected score for a single turn.

This paper will discuss our strategy and implementation of a reinforcement learning based machine learning tool to find the optimal strategy in the game of dice known as Farkle.

## II. BACKGROUND AND RELATED WORK

### A. What is Farkle and how is it played?

Farkle is a standard dice game based on trying to get as many points as possible by the end of the game. The goal is to get the required number of points to win the game (usually 10,000 points). It can be played individually or with multiple players. The game-play is split into rounds which are then split into one or more turns for each of the players for that round. The standard game is played with six six-sided dice though alternate versions of the game can be made to play with  $N$  number of  $M$ -sided dice where  $N, M \in \mathbb{Z} | N > 0, M > 1$ .

The specifics of what happens each round and each turn will be described later in section III-A.

### B. Project Outline Goals

In the Project Outline assignment, we made three tiers of goals for the project.

First, the minimum goal we set was to get a simpler version of Farkle working with the model. What this means is a version of Farkle that works with smaller dice (three-sided dice specifically), and with fewer rules so as to make the implementation and training of the model easier. A fully-functional smaller version of the Farkle model would allow us to eventually set more optimal hyper-parameters like the discount factor or number of training episodes which would in turn help us to develop our next set of goals faster.

Second, the main approach was similar to the minimum goal except to get a fully functional version of Farkle working with a full set of rules. On top of this, we also wanted to increase complexity by adding Q-Learning or tabular MDP. A fully-functional larger version of the Farkle model along with more complex algorithms would not only help us to achieve the main purpose of this project, but also to learn which patterns it has found both for report analysis and also to help with the implementation of our next set of goals.

Third, the stretch goal was to implement an even more optimized and complex version of the model with a heuristic-based opponent modeling system. The goal of this was to try to develop a version of the model that was even better based on human logical reasoning to help save on time for training but also to improve the performance and accuracy.

Analysis of the goals and how they were all met will be located in section VI-A.

### C. Core Concepts

To analyze Farkle computationally, a few foundational game-theory concepts must be clearly defined in the context of our project:

- **Push-Your-Luck Mechanics:** A game design archetype where players must continuously evaluate the optimal stopping point. They are frequently given the option to continue playing to either gain something or risk some loss. Or they could choose to "call it quits" and take the current gain guaranteed. In Farkle, these are represented by either deciding to "Roll" in an effort to possibly gain more scoring dice, but also possibly lose all currently accumulated points for the round. Or they could decide to "Bank" all of their currently accumulated points for the round and end their turn.
- **Competitive Zero-Sum Dynamics:** Competitive Farkle is a zero-sum game where the ultimate reward is binary (win or lose). Since the game terminates when a player reaches the 10,000-point threshold, the opponent's exact score becomes irrelevant once the threshold is crossed. This non-linear reward structure is what dictates the necessity for context-aware state evaluations as it is likely better for a player to play in a more risky way when one or more of their opponents is close to winning.
- **Early Game Risk:** In competitive Farkle, while playing in the early game, players are incentivized to be more risky since they know that they have more time to make back the difference compared to later in the game. These risky actions are also encouraged by the fact that if they "pay off" by granting them many points, then the player can get an early lead over their opponents and have a theoretically safer end game. This adds another layer of complexity to the decision making as it results in an ever changing state of the game depending on what point the game is at. Overall, it makes it so that the players can't just try to get as many points as possible per round. It requires them to think strategically about how they intend to cross the winning threshold specifically before the other players.

### D. Literature Review and Connection

There is a lot of information about how to use RL and MDP modeling on random dice and board games, which provides this research a very strong foundation. Neller and Presser [1] employed Value Iteration to resolve the game of *Pig*, thereby setting a direct precedent for the application of MDPs to push-your-luck mechanics. Tesauro's groundbreaking research on *TD-Gammon* [2] illustrated that Temporal Difference learning could attain superhuman performance in highly stochastic environments. Sutton and Barto [3] formalized the Q-learning algorithm we used, showing that it works well in environments without models.

More specific to the game of Farkle, Busche [4] explored the limitations of static heuristics in Farkle, proving that rigid rule sets fail to adapt to late-game variances and only

result in a small advantage of about 1.775%. Bowling et al. [5] analyzed decision-making in high-variance, imperfect-information environments like Poker, sharing parallels with Farkle's hidden future states. This project directly expands upon Neller and Presser's work [1] by scaling the MDP approach to accommodate the significantly higher multi-dice complexity of Farkle, while introducing competitive state variables to model two-player dynamics.

## III. PROBLEM FORMULATION

### A. Playing the game

First, some notation and game-play patterns. "Game" refers to the period of time from the beginning of a session of game-play until the pre-agreed upon winning condition is met, in this case a score of greater than 10,000 points. If multiple players achieve the winning condition within the same round, the player with the maximum total score wins. If there is still a tie, then the first top player wins. Each "game" is split into "rounds". Within each "round," each player gets a "turn" until all individual players have taken a "turn", after which the next "round" begins. Each player gets one "turn" per "round" at the end of which the points accumulated during the "turn" are added to the individual's total score. "Turn" refers to the period of time from the start of each player's actions for the "round" until an end-turn condition is met. Multiple "choices" can occur in a "turn" and a turn will only end when either a "Farkle" occurs (when the player fails to "roll" any scoring dice) or a player chooses to "bank" and save their accumulated points, thus ending their turn.

Each "choice", the player chooses either "roll" or "bank". If the player chooses to "roll" their remaining dice, and get a "Farkle", then the player's temporary round score is set to zero and their "turn" is over (first of two end-turn conditions). If they "roll" and do get scoring dice, then the score is added to their temporary round score, and the corresponding dice are removed from play (making them frozen dice). If there are still in-play dice after choosing to "roll", then the player gets to make another "choice" with the remaining in-play dice. If the player chooses "bank" then all of their dice become frozen and their temporary turn score is added to their total. If all dice are frozen after a "roll" action is chosen (every die was a scoring die), this is referred to as "hot dice" and all dice are returned to play and another "choice" is made within the "turn". When the player's "turn" is over, their temporary round score is added to their total score. The conditions and point allocation for what counts as scoring dice are described below.

The  $(S, A, T, R, \gamma)$  of the MDP are defined below.

### B. MDP Formulation

We formalize Farkle as a finite Markov Decision Process defined by the tuple  $(S, A, T, R)$ . To manage the complexity of the game, we utilize a discretized state space and a shaped reward function.

- **State Space ( $S$ ):** The environment state is defined by the tuple  $s = (t_b, k_b, d, \delta_b)$ , where:

- $t_b = \lfloor t/150 \rfloor \cdot 150$ : Discretized turn points currently at risk.
- $k_b = \lfloor k/1000 \rfloor \cdot 1000$ : Discretized total banked score of the active agent.
- $d \in \{1, \dots, 6\}$ : The number of dice currently available to roll.
- $\delta_b \in \{-2, -1, 1, 2\}$ : The discretized score differential relative to the leading opponent (Far Behind, Behind, Ahead, Far Ahead).
- **Action Space ( $A$ ):** At any decision node where  $t > 0$ , the agent selects an action  $a \in \{\text{Roll}, \text{Bank}\}$ .
- **Reward Function ( $R$ ):** To encourage optimal risk-taking and competitive awareness, we utilize a shaped reward function rather than raw point values:

$$R(s, a) = \begin{cases} (t/100)^{1.5} & \text{if } a = \text{Bank} \\ -10/d & \text{if } a = \text{Roll and Farkle occurs} \\ 1000 + \Delta_{final} & \text{if player reaches 10,000 pts} \end{cases} \quad (1)$$

The exponential scaling of the banking reward encourages the agent to pursue higher-value turns, while the Farkle penalty is inversely proportional to the number of dice, punishing high-risk rolls with low probability of success.

- **Discount Factor ( $\gamma$ ):**  $\gamma = 0.95$  was determined to be a good value for the discount factor through the development of the minimum goal version of the model. This was achieved through manual iteration through a few common values including  $[1, 0.99, 0.95, 0.9]$ . The use of the discount factor is described in the next few sections such as in the Value Function and the standard Temporal Difference update rule.

### C. Value Function

The objective of the agent is to learn an optimal policy  $\pi^*$  that maximizes the expected cumulative discounted reward. We define the optimal state-value function  $U^*(s)$  using the Bellman Equation:

$$U^*(s) = \max_{a \in A} \left( R(s, a) + \gamma \sum_{s' \in S} T(s' | s, a) U^*(s') \right) \quad (2)$$

Because the exact transition probabilities  $T(s' | s, a)$  of the opponent's future turns are too complex to model identically, we utilize model-free Q-Learning to iteratively approximate the optimal action-value function  $Q^*(s, a)$ .

## IV. SOLUTION APPROACH

### A. Algorithm Description

We implemented Tabular Q-Learning, which is particularly well-suited for environments where the state space can be discretely quantified. The agent updates its quality estimations based on the observed rewards and the maximum anticipated future rewards. The standard Temporal Difference update rule applied is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[ r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (3)$$

### B. Training Pseudocode

```

Initialize  $Q(s, a)$  arbitrarily
for episode in 1 to 5,000,000 do
   $s \leftarrow \text{initial\_state}()$ 
  while  $s$  is not terminal do
    Choose  $a$  using  $\epsilon$ -greedy strategy
    Execute  $a$ , observe  $r$  and  $s'$ 
     $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma \max_{a'} Q(s', a') - Q(s, a))$ 
     $s \leftarrow s'$ 
  end while
end for

```

Notice that for the purposes of training the algorithm, we ended up using five million episodes. We determined that this was the threshold needed to reduce variation in our results by manually and iteratively increasing the number until the results stabilized.

### C. Competitive Greedy Heuristic

As per the stretch goals, we wished to implement a heuristic that used human logic to outperform the Q-Learning model and/or help it to learn and grow. We call the heuristic we came up with, the Competitive Greedy Model. It works by determining both how many points the player has accumulated for the current round so far for their turn, and how far ahead or behind the leading opponent they are. The idea is to play it more safe if they are pretty far ahead of the leading opponent, and conversely, to play more risky if they are fairly far behind the leading opponent. The difference between the current score and the leading opponent's score determines the overall threshold for the round. The threshold is the heuristic's determined limit for how many points the player needs to get in the round before choosing "Bank". If they are below the threshold, then they "Roll", else, they "Bank".

```

Determine, score_diff = my_score - max_opp_score
if score_diff < lowest_setting then
  Loosing badly, threshold = highest
else if score_diff < 2nd_lowest_setting then
  Loosing moderately, threshold = 2nd_highest
  ...
else if score_diff < 2nd_highest_setting then
  Winning moderately, threshold = 2nd_lowest
else
  Winning greatly, threshold = lowest
end if
if (my_score > threshold) OR (my_score == 0) then
  action = "Roll"
else
  action = "Bank"
end if

```

The reason this is called a Competitive Greedy heuristic is because, while it has similarities to a greedy heuristic, it is ultimately not a greedy heuristic for Farkle. This is because a truly greedy heuristic for Farkle is one that would always choose to "Roll" since that is the only way to get more points. However, a human player would still care about their

current standing in a competitive game of Farkle meaning that they would be more likely to "Bank" the further ahead they are. However, given the simplicity of this heuristic, and it's tendency to choose "Roll" especially if it is losing, it should still be considered close to a greedy heuristic. Therefore, Competitive Greedy heuristic.

#### D. Implementation Choices

- **Exploration vs. Exploitation:** We utilized an  $\epsilon$ -greedy strategy with linear decay. The exploration rate  $\epsilon$  decayed from 1.0 to 0.01 over 4,000,000 episodes (80% of total training time) to ensure the agent explored the vast state-action pairs generated by the multi-dice scoring combinations.
- **State Space Reduction:** Unmodified, the state space for a game to 10,000 points is computationally prohibitive. By implementing the *diff\_bucket* logic and point discretization described in Section III, we reduced the effective state space to approximately 17,181 reachable state-action pairs, allowing for convergence within the tabular Q-dictionary.
- **Dynamic Opponent Training:** Unlike standard models that train against random agents, our agent was trained against a *Competitive Greedy* baseline. This baseline adjusts its own banking threshold (ranging from 150 to 600 points) based on the score differential, forcing the RL agent to learn defensive and offensive adaptations.
- **Reward Boosts/Reductions:** As an extra layer of precaution to ensure that the model is trying to learn how to win, extra bonuses are added to the current reward (not the current points) if the model wins before the opponent. The same thing also happens in reverse where the current reward is reduced if the opponent wins before the player.

### V. RESULTS

#### A. Experimental Setup

As mentioned in section IV-C, the Q-Learning agent was evaluated against a **Competitive Greedy Heuristic**. Unlike a static bot, this heuristic employs a sliding threshold: it banks at 150 points when ahead to minimize variance, and takes risks up to 600 points when trailing significantly. This represents a rigorous benchmark that mimics human-like risk adjustment. Performance was measured over 10,000 matches to a target score of 10,000.

TABLE I  
AGENT PERFORMANCE AGAINST COMPETITIVE HEURISTIC

| Agent              | Win Rate (%) | Strategy Description        |
|--------------------|--------------|-----------------------------|
| Aware Q-Learning   | 58.2%        | Adaptive (Bucketed MDP)     |
| Competitive Greedy | 41.8%        | Dynamic Threshold Heuristic |

#### B. Analysis

The Q-Learning agent achieved a 58.2% win rate. The 16.4% performance margin over the dynamic heuristic is notable; it suggests that the RL agent discovered optimal

"Roll" conditions that the heuristic's linear thresholds could not capture—specifically regarding the transition to "Hot Dice" states. Analysis of the Q-table confirms that the agent prioritizes "Roll" actions even at high point totals if  $d = 6$ , effectively exploiting the lower probability of a Farkle on a full set of dice.

### VI. DISCUSSION AND CONCLUSIONS

#### A. Discussion of Project Outline Goals and Relationship to Class Topics

As seen in this report, we not only achieved the minimum and main approach goals, we also achieved an implemented version of the stretch goals. The minimum goals were met through the initial creation of the model, shown by the fact that a fully-functional large version of the model worked throughout the entirety of the report, and training of hyper-parameters. The main approach goals were achieved through the creation of the fully-functional model with Q-Learning that used the hyper-parameters that were found during the minimum goal development stage. Finally, the stretch goals were met with the creation of the Competitive Greedy heuristic. Which, while this wasn't a particularly complex heuristic and while this heuristic didn't outperform the Q-Learning model. It still helped to show us some of the patterns that emerged from the Q-Learning model and how it performs in our simulated version of Farkle.

As for the relationship to the class. This project clearly demonstrates knowledge of machine learning, specifically reinforcement learning, via the use of topics like Q-Learning, greedy policies (even if only a partial greedy policy), and Bellman Operators along with the overall structure of the MDP problem defined above in section III-B.

#### B. Conclusions

The application of Tabular Q-Learning to Farkle successfully yielded an agent capable of dynamic, context-aware decision-making. By formalizing the push-your-luck and competitive zero-sum game mechanics as an MDP, the agent learned to seamlessly adjust its risk tolerance. The data clearly shows that static heuristic approaches—such as banking strictly at a set point value—are suboptimal, as they fail to account for the shifting utility of points based on available dice  $d$  and competitive standing  $b$ .

#### C. Limitations and Future Work

While the *diff\_bucket* methodology successfully curbed state-space explosion, discretization inherently introduces edge-case inefficiencies (e.g., treating a 10-point lead identical to a 490-point lead within the same bucket). Future work will aim to transcend tabular limitations by implementing Deep Q-Networks (DQN), allowing for continuous state representations. Additionally, introducing multi-agent self-play mechanisms will be explored to move beyond exploiting static heuristics and attempt to approximate a Nash Equilibrium for competitive Farkle strategy.

## VII. REQUIRED ITEMS

### A. Contributions

Nathan Cohen developed most of the program, trained the model, and edited the report.

Robert Kaplan wrote most of the report, developed initial ideas, provided support for the code.

### B. Public Sharing Policy

The authors grant permission for this report to be posted publicly.

### C. Program Implementations

AI models such as Gemini and Claude were used to produce the initial Farkle game simulator in Julia since there was no existing model that fit our needs and since the creation of a game simulator was not in the scope of this course. The rest of the code, including and implementing the q-Learning algorithms and greedy models, was created largely by Nathan Cohen and reviewed by Robert Kaplan.

## REFERENCES

- [1] T. Neller and C. Presser, "Optimal Play of the Dice Game Pig," The UMAP Journal, 2004.
- [2] G. Tesauro, "Temporal Difference Learning and TD-Gammon," Communications of the ACM, 1995.
- [3] R. Sutton and A. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
- [4] M. Busche and T. Neller, "Optimal Play of the Farkle Dice Game," Gettysburg College Department of Computer Science, 2017.
- [5] M. Bowling et al., "Heads-up Limit Hold'em Poker is Solved," Science, 2015.

## VIII. APPENDIX

```
using Random
using Serialization
using StatsBase

# --- Farkle ENGINE ---

mutable struct GameState
    scores::Vector{Int}
    current_player::Int
    p::Int
    n::Int
end

function roll_dice(n::Int, N::Int=6)
    return rand(1:N, n)
end

function calculate_score(roll::Vector{Int},
    ↪ N::Int=6)
    score = 0
    scoring_dice = 0
    n_dice = length(roll)
    counts = zeros{Int, N}

    for die in roll
        if die <= N
            counts[die] += 1
        end
    end
```

```
        end
    end

    if n_dice == 6
        if count(x -> x == 3, counts) == 2 return
            ↪ 2500, 6 end
        if count(x -> x == 2, counts) == 3 return
            ↪ 1500, 6 end
        if count(x -> x == 4, counts) == 1 &&
            ↪ count(x -> x == 2, counts) == 1 return
            ↪ 1500, 6 end
        if all(counts[1:N] .== 1) return 1500, 6 end
    end

    has_partial = true
    for i in 1:(N-1)
        if counts[i] < 1
            has_partial = false
            break
        end
    end

    if has_partial
        score += 750
        scoring_dice += (N - 1)
        for i in 1:(N-1)
            counts[i] -= 1
        end
    end

    for die in 1:N
        c = counts[die]
        if c == 6
            score += 3000; scoring_dice += 6;
            ↪ counts[die] -= 6
        elseif c == 5
            score += 2000; scoring_dice += 5;
            ↪ counts[die] -= 5
        elseif c == 4
            score += 1000; scoring_dice += 4;
            ↪ counts[die] -= 4
        elseif c == 3
            score += (die == 1 ? 1000 : die * 100)
            scoring_dice += 3; counts[die] -= 3
        end
    end

    if counts[1] > 0
        score += counts[1] * 50; scoring_dice +=
            ↪ counts[1]
    end

    if N >= 5 && counts[5] > 0
        score += counts[5] * 100; scoring_dice +=
            ↪ counts[5]
    end

    return score == 0 ? (0, 0) : (score,
        ↪ scoring_dice)
end

# Helper function to find the highest score among
↪ ALL OTHER players
```

```

function get_max_opponent_score(state::GameState)
    if length(state.scores) == 1
        return 0 # Solo mode fallback
    end

    opp_scores = [state.scores[i] for i in
        ↪ 1:length(state.scores) if i !=
        ↪ state.current_player]
    return maximum(opp_scores)
end

# --- GAME LOOP ---

function
    ↪ play_game(player_policies::Vector{Function},
    ↪ target_score::Int=2000, print_logs::Bool=true)
    num_players = length(player_policies)
    state = GameState(zeros{Int, num_players}, 1,
        ↪ 0, 6)
    round_number = 1

    if print_logs
        println(
            ↪ "=== Starting Competitive Farkle with ↪
            ↪ $num_players Player(s) ===")
    end

    while all(state.scores .< target_score)
        if print_logs
            println("\n--- Round $round_number ---")
        end

        for i in 1:num_players
            state.current_player = i
            state.p = 0
            state.n = 6
            is_turn_active = true

            if print_logs
                println("\n>> Player $i's turn!")
            end

            while is_turn_active
                b_current = state.scores[i]
                max_opp =
                    ↪ get_max_opponent_score(state)
                if print_logs
                    println("State: [Your Bank: ↪
                        ↪ $b_current ↪
                        ↪ | Highest Opponent: $max_opp ↪
                        ↪ | Turn Points: $(state.p) ↪
                        ↪ | Dice Left: $(state.n)]")
                end

                action = player_policies[i](state,p)
                ↪ rint_logs)

                if action == "Bank"
                    state.scores[i] += state.p
                    if print_logs

```

```

                println("-> Banked! Player $i ↪
                    ↪ total score is now $(sta ↪
                    ↪ te.scores[i]).")
                end
                is_turn_active = false

                if state.scores[i] >=
                    ↪ target_score
                    if print_logs
                        println("\n*** Player $i ↪
                            ↪ has crossed the target score! ***")
                            ↪ )
                    end
                end

            elseif action == "Roll"
                roll = roll_dice(state.n)
                if print_logs
                    println("-> Rolled: $roll")
                end

                score, used_dice =
                    ↪ calculate_score(roll)

                if score == 0
                    if print_logs
                        println("-> FARKLE! Player ↪
                            ↪ $i ↪
                            ↪ lost their turn points." ↪
                            ↪ )
                    end
                    state.p = 0
                    is_turn_active = false
                else
                    state.p += score
                    state.n -= used_dice
                    if print_logs
                        println("    Scored $score ↪
                            ↪ points on that roll.")
                    end

                    if state.n == 0
                        if print_logs
                            println(
                                ↪ "    Hot dice! 6 new dice." ↪
                                ↪ )
                            end
                        state.n = 6
                    end
                end

            elseif action == "Quit"
                println("\nPlayer $i ↪
                    ↪ chose to quit. Ending the game early.. ↪
                    ↪ )
                return # This immediately exits
                    ↪ the entire play_game
                    ↪ function
            end
        end

        if maximum(state.scores) >= target_score
            break

```

```

        end
    end
    round_number += 1
end

winner = argmax(state.scores)
winning_score = state.scores[winner]
if print_logs
    println("\n=== Game Over! Player $winner\
    ↪ wins with $winning_score points! ===")
end
return winner
end

# --- COMPETITIVE PLAYER POLICIES ---

function human_player(state::GameState, print_logs::Bool=true)
    while true
        # Change the prompt based on whether a
        ↪ player has points to bank
        if state.p == 0
            print("Choose action (Roll / Quit): ")
        else
            print(
                ↪ "Choose action (Roll / Bank / Quit): "
            )
        end

        action = titlecase(strip(readline()))

        # Prevent banking 0 points
        if state.p == 0 && action == "Bank"
            println(
                ↪ "You cannot bank with 0 points. Please
                ↪ )
            continue
        end

        # Accept valid commands
        if action in ["Roll", "Bank", "Quit"]
            return action
        end

        println(
            ↪ "Invalid input. Please type 'Roll', 'Bank',
            ↪ )
    end
end

# A smarter greedy agent that adjusts its risk
↪ tolerance based on opponents
function competitive_greedy_player(state::GameState,
    ↪ , print_logs::Bool=true)
    my_score = state.scores[state.current_player]
    max_opp_score = get_max_opponent_score(state)
    score_diff = my_score - max_opp_score

    # Adjust required points to bank based on how
    ↪ far ahead/behind they are
    threshold = 250
    if score_diff < -500

```

```

        threshold = 600 # Losing badly: Take massive
        ↪ risks
    elseif score_diff < -200
        threshold = 400 # Losing: Take moderate
        ↪ risks
    elseif score_diff > 500
        threshold = 150 # Winning: Play it very safe
    end

    if state.p == 0
        return "Roll"
    elseif state.p < threshold
        if print_logs
            println("Agent (Threshold $threshold\
            ↪ ) decides to: Roll")
        end
        return "Roll"
    else
        if print_logs
            println("Agent (Threshold $threshold\
            ↪ ) decides to: Bank")
        end
        return "Bank"
    end
end

# --- AWARE Q-LEARNING AGENT ---

# Helper function to bucket the score difference to
↪ prevent a state space explosion
function get_diff_bucket(my_score::Int,
    ↪ max_opp_score::Int)
    diff = my_score - max_opp_score
    if diff <= -500 return -2 # Far behind
    elseif diff < 0 return -1 # Slightly
        ↪ behind
    elseif diff < 500 return 1 # Slightly ahead
    else return 2 # Far ahead
    end
end

function bucket_turn_points(p::Int)
    # Group turn points into buckets of 150 to
    ↪ compress the state space
    return (p ÷ 150) * 150
end

function bucket_banked_points(b::Int)
    # Group banked points into buckets of 1000
    return (b ÷ 1000) * 1000
end

mutable struct AwareQLearningAgent
    # Tuple is now: (Turn Points, Banked, Dice, Diff
    ↪ Bucket, Action)
    Q::Dict{Tuple{Int, Int, Int, Int, String},
        ↪ Float64}
    alpha::Float64
    gamma::Float64
    epsilon::Float64
end

```

```

function AwareQLearningAgent(; alpha=0.1,
↳ gamma=0.95, epsilon=1.0)
    return AwareQLearningAgent(Dict{Tuple{Int, Int,
↳ Int, Int, String}, Float64}{}), alpha,
↳ gamma, epsilon)
end

function get_q(agent::AwareQLearningAgent, p::Int,
↳ b::Int, n::Int, diff_bucket::Int,
↳ action::String)
    return get(agent.Q, (p, b, n, diff_bucket,
↳ action), 0.0)
end

function choose_action(agent::AwareQLearningAgent,
↳ state::GameState; explore=true)
    if state.p == 0 return "Roll" end

    b_current = state.scores[state.current_player]
    max_opp_score = get_max_opponent_score(state)

    # Compress the state into buckets exactly like
    ↳ in training
    p_bucket = bucket_turn_points(state.p)
    b_bucket = bucket_banked_points(b_current)
    diff_bucket = get_diff_bucket(b_current,
↳ max_opp_score)

    if explore && rand() < agent.epsilon
        return rand(["Roll", "Bank"])
    end

    # Query the Q-table using the buckets
    q_roll = get_q(agent, p_bucket, b_bucket,
↳ state.n, diff_bucket, "Roll")
    q_bank = get_q(agent, p_bucket, b_bucket,
↳ state.n, diff_bucket, "Bank")

    return q_roll > q_bank ? "Roll" : "Bank"
end

function save_agent(agent::AwareQLearningAgent,
↳ filename::String="q_agent.jls")
    serialize(filename, agent.Q)
    println("Agent Q-table saved to $filename")
end

function load_agent!(agent::AwareQLearningAgent,
↳ filename::String="q_agent.jls")
    if isfile(filename)
        agent.Q = deserialize(filename)
        println("Successfully loaded Q-table from
↳ $filename with $(length(agent.Q))
↳ state-action pairs.")
        return true
    else
        println("No saved agent found at $filename
↳ .")
        return false
    end
end

```

```

# Updated training function to train against greedy
↳ agent for max win rate
function train_agent!(agent::AwareQLearningAgent,
↳ opponent_policy::Function,
↳ episodes::Int=100_000, target_score::Int=10000)
    println(|
↳ "Training Q-Learning Agent against opponent for
↳ $episodes episodes...")

    for ep in 1:episodes
        # Decay exploration rate over time
        agent.epsilon = max(0.01, 1.0 - (ep /
↳ (episodes * 0.8)))

        # Set up a real 2-player game (Player 1:
        ↳ Agent, Player 2: Opponent)
        state = GameState([0, 0], 1, 0, 6)

        while maximum(state.scores) < target_score
            # - AGENT'S TURN -
            state.current_player = 1
            state.p = 0
            state.n = 6
            is_agent_turn = true

            while is_agent_turn
                # Observe current state
                p_current = state.p
                b_current = state.scores[1]
                n_current = state.n
                opp_score = state.scores[2]

                # Apply bucketing to compress state
                ↳ space
                p_bucket =
                ↳ bucket_turn_points(p_current)
                b_bucket =
                ↳ bucket_banked_points(b_current)
                diff_bucket =
                ↳ get_diff_bucket(b_current,
                ↳ opp_score)

                # Choose action
                action = choose_action(agent,
                ↳ state, explore=true)

                # Variables to track the outcome of
                ↳ the action
                reward = 0.0
                turn_ended = false

                # Execute action
                if action == "Bank"
                    state.scores[1] += state.p
                    reward = (state.p / 100.0)^1.5 #
                    ↳ exponential reward for
                    ↳ banking big
                    turn_ended = true
                elseif action == "Roll"
                    roll = roll_dice(state.n)

```



```

score, used_dice =
  ↪ calculate_score(roll)

if score == 0
  # A steep penalty for rolling
  ↪ with fewer dice
  reward = reward = -10.0 /
  ↪ state.n
  state.p = 0
  turn_ended = true
else
  state.p += score
  state.n -= used_dice
  if state.n == 0 state.n = 6
  ↪ end
end
end

# When the agent wins:
if state.scores[1] >= target_score
  score_diff = state.scores[1] -
  ↪ state.scores[2]
  reward += 1000.0 + score_diff
  turn_ended = true
end

# - OPPONENT'S TURN (Simulated if
  ↪ Agent's turn ended and game
  ↪ isn't over) -
if turn_ended && state.scores[1] <
  ↪ target_score
  state.current_player = 2
  state.p = 0
  state.n = 6
  is_opp_turn = true

while is_opp_turn
  # Simulate the greedy bot
  ↪ (pass false to suppress
  ↪ logs)
  opp_action =
  ↪ opponent_policy(state,
  ↪ false)

  if opp_action == "Bank"
    state.scores[2] +=
    ↪ state.p
    is_opp_turn = false
  elseif opp_action == "Roll"
    opp_roll =
    ↪ roll_dice(state.n)
    opp_score_roll,
    ↪ opp_used = calculat
    ↪ e_score(opp_roll)
    if opp_score_roll == 0
      state.p = 0
      is_opp_turn = false
    else
      state.p +=
      ↪ opp_score_roll

```

```

state.n -= opp_used
if state.n == 0
  ↪ state.n = 6 end
end
end

# Reset turn state for the
  ↪ agent's next observation
state.current_player = 1
state.p = 0
state.n = 6

# When the opponent wins
  ↪ (simulated turn):
if state.scores[2] >= target_score
  score_diff = state.scores[2] -
  ↪ state.scores[1]
  reward -= (1000.0 + score_diff)
end
end

# Observe Next State & Update
  ↪ Q-Table
p_next_bucket =
  ↪ bucket_turn_points(state.p)
b_next_bucket = bucket_banked_point
  ↪ s(state.scores[1])
next_diff_bucket = get_diff_bucket(
  ↪ state.scores[1],
  ↪ state.scores[2])

# Determine max Q for the next state
q_next_roll = get_q(agent,
  ↪ p_next_bucket, b_next_bucket,
  ↪ state.n, next_diff_bucket,
  ↪ "Roll")
q_next_bank = state.p == 0 ? -Inf :
  ↪ get_q(agent, p_next_bucket,
  ↪ b_next_bucket, state.n,
  ↪ next_diff_bucket, "Bank")

# If the game is over, there is no
  ↪ future state
max_q_next = maximum(state.scores)
  ↪ >= target_score ? 0.0 :
  ↪ max(q_next_roll, q_next_bank)

# Fetch previous knowledge using the
  ↪ buckets!
current_q = get_q(agent, p_bucket,
  ↪ b_bucket, n_current,
  ↪ diff_bucket, action)

# Apply Bellman Equation
new_q = current_q + agent.alpha *
  ↪ (reward + agent.gamma *
  ↪ max_q_next - current_q)

# Save the updated knowledge back to
  ↪ the same bucket key

```

```

        agent.Q[(p_bucket, b_bucket,
        ↪ n_current, diff_bucket,
        ↪ action)] = new_q
        if turn_ended
            is_agent_turn = false
        end
    end
end
end
println("Training complete. Q-Table contains $(
    ↪ length(agent.Q)) state-action pairs.")
end

function
    ↪ trained_q_policy(agent::AwareQLearningAgent)
        return (state::GameState, print_logs::Bool=true)
        ↪ -> begin
            action = choose_action(agent, state,
            ↪ explore=false)
            if print_logs
                println("Q-Agent decides to: $action")
            end
            return action
        end
    end
end

# --- EXECUTION OF ACTUAL GAME ---

# Initialize the aware RL agent
my_agent = AwareQLearningAgent()
model_filename = "q_agent.jls"

# Try to load the agent. If it fails, train it and
    ↪ save it
if load_agent!(my_agent, model_filename)
    println(
        ↪ "Skipping training. Using pre-trained agent."
        ↪ )
else
    training_episodes = 5_000_000
    # Pass the competitive greedy player into the
        ↪ training function
    train_agent!(my_agent,
        ↪ competitive_greedy_player,
        ↪ training_episodes, 10000)
    save_agent(my_agent, model_filename)
end

q_bot = trained_q_policy(my_agent)

function get_percentages(arr, player_names)
    n = length(arr)
    counts = countmap(arr)

    # Use player_names[val] to map the ID to the
        ↪ name
    return Dict{player_names[val] =>
        ↪ "$(round((count / n) * 100, digits=1))%"
            for (val, count) in counts}
end

```

```

# Game including human for testing
println("\n\n--- MATCH: Q-Agent (Player 1) vs.
    ↪ Competitive Greedy Agent (Player 2) vs. Human
    ↪ (Player 3)---")
play_game([q_bot, competitive_greedy_player,
    ↪ human_player], 10000)

# Game loop without a human for many runs to
    ↪ estimate win rates.
# 1000 matches:
matches = 10000
scores = zeros{Int, matches}
for i in 1:matches
    scores[i] = play_game([q_bot,
        ↪ competitive_greedy_player], 10000, false)
end

results = get_percentages(scores, ["q_bot",
    ↪ "competitive_greedy_player"])
println(results)

```